Лабораторна робота №1

Частина 1

Огляд інструментів для роботи з AI: налаштування середовища (Python, TensorFlow, PyTorch, Google Colab). Створення базового класифікатора рукописних цифр MNIST для прикладу в навчанні

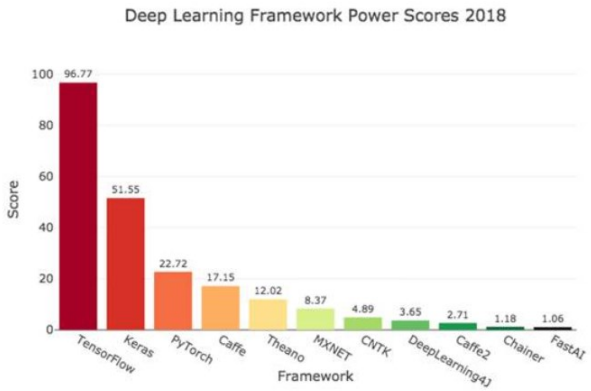
Мета роботи

Налаштувати робоче середовище для розробки моделей ШІ (Python + бібліотеки, Google Colab) та створити базовий класифікатор рукописних цифр MNIST як референтний приклад, що демонструє повний цикл: підготовка даних → побудова простої нейромережі (на прикладі CNN) у TensorFlow/Keras (або PyTorch) → навчання й оцінювання якості (accuracy, confusion matrix) → відтворюваний запуск.

Теоретичний матеріал

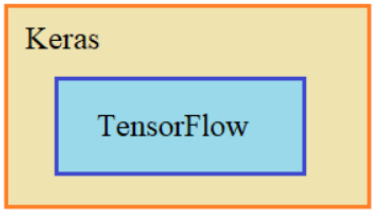
TensorFlow – один із найвідоміших фреймворків для машинного та глибинного навчання, розроблений командою Google мовою програмування C++ і представлений у 2015 році. Він надає засоби для створення, навчання, оптимізації та розгортання нейронних мереж різної архітектури, а також підтримує виконання обчислень на центральних і графічних процесорах (CPU та GPU) і спеціалізованих прискорювачах.

На рисунку наведено діаграму популярності фреймворків глибинного навчання станом на 2018 рік. Вона відображає один з етапів розвитку відповідних програмних екосистем і демонструє провідні позиції TensorFlow у цей період.



У перших версіях TensorFlow побудова моделей значною мірою ґрунтувалася на явному формуванні обчислювальних графів, що забезпечувало гнучкість і можливості оптимізації, але водночас ускладнювало програмну реалізацію моделей. Для спрощення розроблення нейронних мереж набув поширення високорівневий API Keras, який надає зручні засоби для послідовного формування шарів, налаштування параметрів навчання та проведення експериментів.

У сучасних версіях TensorFlow Keras використовується як основний високорівневий інтерфейс для створення нейронних мереж. Завдяки цьому розробник може зосередитися передусім на архітектурі моделі та процесі її навчання, не реалізуючи вручну більшість низькорівневих операцій.



У даному розділі розглянемо інтерфейс TensorFlow для мови Python, інтегрований у фреймворк Keras, особливості побудови моделей, а також інструменти їх візуалізації за допомогою TensorBoard та comet ml.

Архітектура TensorFlow ґрунтується на низці ключових принципів, що роблять його потужним інструментом для глибинного навчання (DL) та різноманітних чисельних обчислень.

- **Тензори.** Основною структурою даних у TensorFlow є тензор – багатовимірний масив. Саме в такій формі подаються як вхідні дані (наприклад, зображення), так і параметри моделей (ваги, зміщення тощо).
- **Обчислювальні графи.** Спочатку TensorFlow використовував статичний обчислювальний граф для визначення послідовності операцій. У сучасних версіях за замовчуванням застосовується режим *Eager Execution*, що забезпечує більш інтуїтивну, «пітонівську» взаємодію з бібліотекою. Проте графова модель залишається важливою завдяки можливостям оптимізації та ефективного розгортання, оскільки вона дає змогу компілювати обчислення й виконувати їх із максимальною продуктивністю на GPU та TPU.
- **Масштабованість.** TensorFlow спроєктовано для навчання та інференсу у великих розподілених системах. Він здатний працювати як на окремому процесорі, так і на кластерах графічних процесорів чи спеціалізованих прискорювачах, що робить його придатним як для наукових досліджень, так і для промислового застосування.
- **Екосистема.** TensorFlow – це не лише бібліотека для машинного навчання, а й широка екосистема інструментів. До неї входять TensorBoard для візуалізації процесів навчання, TensorFlow Serving для високопродуктивного розгортання моделей, а також TensorFlow Lite – рішення для використання моделей на мобільних та вбудованих пристроях.

TensorFlow завдяки розвинутим інструментам для побудови, оптимізації та розгортання моделей історично набув широкого застосування у промислових системах. Натомість PyTorch став особливо популярним у науковому та дослідницькому середовищі завдяки простоті, гнучкості та зручності експериментування з новими архітектурами. У 2025–2026 роках PyTorch широко застосовується у наукових дослідженнях, розробленні LLM та екосистемі Hugging Face, а також широко використовується як основна платформа для реалізації сучасних генеративних моделей. Водночас упровадження в TensorFlow режиму *Eager Execution*, який забезпечує безпосереднє виконання операцій і спрощує налагодження коду, зробило роботу з цією платформою значно зручнішою та певною мірою скоротило розрив між двома фреймворками. Тому вибір між TensorFlow і PyTorch зазвичай визначається особливостями конкретного проєкту, вимогами до подальшого розгортання моделі та досвідом розробників у відповідній екосистемі.

Keras виступає високорівневим API для побудови нейронних мереж і нині офіційно інтегрований у TensorFlow як стандартний інтерфейс високого рівня. Його головною перевагою є інтуїтивність та зручність у конструюванні моделей, оскільки він приховує більшість технічних деталей низького рівня. Для значної частини розробників робота з TensorFlow фактично означає використання API `tf.keras`.

Щоб встановити поточну версію, яка включає підтримку графічних карт із CUDA (Ubuntu та Windows):

```
$ pip install tensorflow
```

Інші пристрої (DirectX та MacOS-metal) підтримуються за допомогою Device Plugins.

Також доступний компактніший пакет лише для процесора (CPU):

```
$ pip install tensorflow-cpu
```

Для оновлення TensorFlow до останньої версії необхідно додати прапорець `--upgrade` до наведених вище команд.

Можливо, також знадобиться встановити Keras як окремий пакет:

```
$ pip install keras
```

Модуль `tf.keras`

Даний модуль містить інструменти для побудови моделей, зокрема:

- Шари (layers): серед яких повнозв'язний (`Dense`), згортковий (`Conv1D`), рекурентний (`RNN, LSTM`) та інші. Загалом доступно 105 шарів.
- Функції активації (activations): такі як `Softmax, ReLU, Sigmoid`. Деякі з них, наприклад `PReLU`, реалізовані у вигляді окремих шарів. Загалом передбачено 15 функцій активації.
- Функції втрат (losses): включаючи `Binary Crossentropy, MAE, MSE`. Усього доступно 25 функцій втрат.
- Метрики (metrics): наприклад, `MAE, MSE, Cosine Similarity`. Реалізовані як 43 класи та 25 функцій.
- Оптимізатори (optimizers): серед них `SGD, Adam, Adagrad`. Усього передбачено 9 оптимізаторів.
- Зворотні виклики (callbacks): наприклад, `RemoteMonitor` для передачі подій на сервер, `History` для відстеження історії навчання, `EarlyStopping` для зупинки тренування за відсутності покращень, `TensorBoard` для візуалізації. Загалом доступно 14 callback-функцій.

Побудова моделей

У TensorFlow існує два основні підходи до створення моделей.

1. **Послідовне додавання блоків (Sequential API).** У цьому випадку модель формується як конструктор шляхом накладання шарів один на один. Розробник покладається на вбудовані реалізації алгоритмів прямого та зворотного проходів.

Прикладом є нейронна мережа на Python із використанням повнозв'язних шарів, яка ілюструє блочний підхід до побудови:

```
import tensorflow as tf

# Побудова послідовної моделі
model = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=(shape_0, shape_1)),
# Перетворення вхідних даних у вектор
    tf.keras.layers.Dense(256, activation="relu"),
# Прихований шар із 256 нейронів
    tf.keras.layers.Dropout(0.2),
# Dropout для зменшення перенавчання
    tf.keras.layers.Dense(10, activation="softmax")
# Вихідний шар для 10 класів
```

```
])

# Компіляція моделі
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Навчання моделі
model.fit(x_train, y_train, epochs=5, batch_size=32, validation_split=0.1)

# Оцінювання моделі
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"\nТочність на тестових даних: {test_acc:.4f}")
```

2. **Побудова моделей за допомогою субкласування.** Другий підхід до створення моделей у TensorFlow орієнтований на досвідчених спеціалістів з машинного навчання. Він передбачає безпосереднє **субкласування класу tf.keras.Model**, що дає змогу розробнику самостійно визначати шари, функції активації та логіку проходження даних крізь модель. Такий метод забезпечує максимально гнучкий контроль над архітектурою, даючи змогу будувати нетипові рішення, експериментувати з нестандартними шарами чи процедурами навчання. Водночас він потребує глибшого розуміння API tf.keras, оскільки більшість процесів (наприклад, визначення прямого проходу) реалізується вручну.

Нижче наведено приклад створення згорткової нейронної мережі (Convolutional Neural Network) із використанням субкласування:

```
import tensorflow as tf
from tensorflow.keras.layers import Conv2D, Flatten, Dense

class MyModel(tf.keras.Model):
    def __init__(self):
        super(MyModel, self).__init__()
        self.conv1 = Conv2D(64, 3, activation="relu")
        self.flatten = Flatten()
        self.d1 = Dense(128, activation="relu")
        self.d2 = Dense(10, activation="softmax")

    def call(self, x):
        x = self.conv1(x)
        x = self.flatten(x)
        x = self.d1(x)
        return self.d2(x)
# створення моделі
model = MyModel()

# навчання з використанням GradientTape
with tf.GradientTape() as tape:
    logits = model(images)
    loss_value = loss(logits, labels)

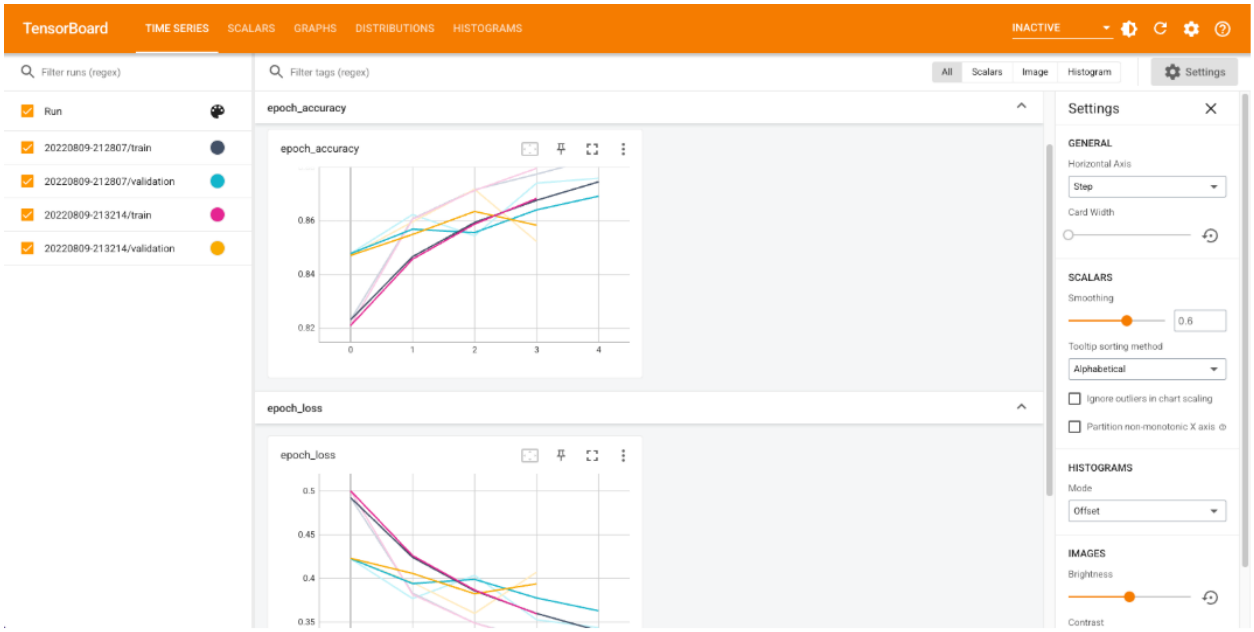
grads = tape.gradient(loss_value, model.trainable_variables)
optimizer.apply_gradients(zip(grads, model.trainable_variables))
```

TensorFlow має власний інструмент для візуалізації – TensorBoard. За його допомогою можна:

- відстежувати зміни точності (*accuracy*) та функції втрат (*loss*) у реальному часі;
- візуалізувати граф моделі з шарами та операціями;
- переглядати параметри моделі (ваги, зміщення) та інші тензори, які змінюються в процесі навчання;
- виводити зображення, відео або аудіо;
- та виконувати багато інших завдань.

Запустити TensorBoard (https://www.tensorflow.org/tensorboard/) можна як на локальному комп'ютері (*localhost*), так і в середовищах Jupyter Notebook (https://jupyter.org/) чи Google Colab (https://colab.google/).

На рисунку нижче подано приклад інтерфейсу TensorBoard.



Розпізнавання цифр із набору MNIST

На початковому етапі лабораторної роботи передбачається створення та навчання згорткової нейронної мережі (CNN), призначеної для класифікації рукописних цифр із класичного датасету MNIST. Цей набір містить 60 000 навчальних та 10 000 тестових зображень, що охоплюють цифри від 0 до 9.

Перед побудовою згорткової нейронної мережі розглянемо простішу повнозв'язну модель. Її мета — продемонструвати базовий принцип створення, компіляції, навчання та оцінювання нейронної мережі в TensorFlow/Keras. Надалі цей самий загальний шаблон буде використано для побудови CNN, але архітектура моделі буде доповнена згортковими шарами, які дають змогу враховувати просторову структуру зображення.

Перед початком роботи слід інсталювати необхідні бібліотеки та виконати імпорт відповідних модулів.

```
# Імпорт TensorFlow
# !pip install tensorflow
```

```
import tensorflow as tf

# Інші пакети
import matplotlib.pyplot as plt
import numpy as np
import random
```

Опис пакетів:

- tensorflow – основна бібліотека для побудови та навчання нейронних мереж.
- matplotlib.pyplot – для візуалізації графіків і результатів.
- numpy – для роботи з масивами та математикою.
- random – для генерації випадкових чисел.

Також встановимо Comet**.** Comet ML — це платформа для відстеження ML-експериментів, яка дає змогу зберігати гіперпараметри, метрики, графіки й результати різних запусків та порівнювати їх.

Попередньо необхідно створити обліковий запис Comet та ввести API-ключ нижче.

```
# Встановлення Comet ML
!pip install comet_ml > /dev/null 2>&1
import comet_ml
# Авторизація в Comet ML
# Під час першого запуску система запропонує ввести API-ключ
comet_ml.login()
# Перевірка наявності GPU
gpus = tf.config.list_physical_devices("GPU")
if gpus:
    print("GPU доступний:", gpus)
else:
    print("GPU не виявлено. Навчання виконуватиметься на CPU.")
# Створення нового експерименту Comet ML
comet_model_1 = comet_ml.start(
    project_name="lab1_part1_NN"
)
# Збереження основних параметрів експерименту
comet_model_1.log_parameters({
    "batch_size": BATCH_SIZE,
    "epochs": EPOCHS,
    "optimizer": "Adam",
    "learning_rate": 0.001
})
```

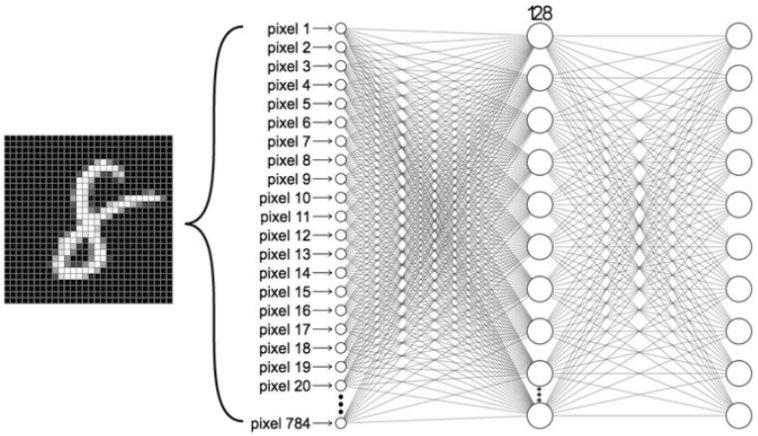
Завантажимо набір даних і виведемо кілька випадкових прикладів із нього:

```
mnist = tf.keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) =
mnist.load_data()
train_images = (np.expand_dims(train_images, axis=-
1)/255.).astype(np.float32)
train_labels = (train_labels).astype(np.int64)
test_images = (np.expand_dims(test_images, axis=-
1)/255.).astype(np.float32)
test_labels = (test_labels).astype(np.int64)
```

Тренувальний набір складається з 28x28 зображень відтінків сірого з рукописними цифрами. Візуалізуємо деякі з цих зображень та їхні відповідні тренувальні мітки.

```
plt.figure(figsize=(10,10))
random_inds = np.random.choice(60000,36)
for i in range(36):
    plt.subplot(6,6,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    image_ind = random_inds[i]
    plt.imshow(np.squeeze(train_images[image_ind]), cmap=plt.cm.binary)
    plt.xlabel(train_labels[image_ind])
comet_model_1.log_figure(figure=plt)
```

На початковому етапі створимо найпростішу модель нейронної мережі, яка міститиме два шари з повним з'єднанням. Вона застосовуватиметься для розв'язання задачі класифікації рукописних цифр. Результатом роботи мережі буде ймовірнісний розподіл між десятьма можливими класами (0–9). Схематичне зображення цієї базової архітектури наведено нижче.




```
def build_fc_model():
    # Побудова простої повнозв'язної нейронної мережі (Fully Connected NN)
    fc_model = tf.keras.Sequential([
        # Шар Flatten для перетворення вхідних даних у вектор
        tf.keras.layers.Flatten(),

        # Перший прихований шар із 128 нейронами та функцією активації ReLU
        tf.keras.layers.Dense(128, activation="relu"),

        # Вихідний шар із 10 нейронами (для 10 класів), функція активації
Softmax
        tf.keras.layers.Dense(10, activation="softmax")
    ])
    return fc_model

# Створення моделі
model = build_fc_model()
```

Створена нейронна мережа має просту та логічну структуру. На початковому етапі застосовується шар tf.keras.layers.Flatten, який виконує перетворення вхідних даних: двовимірне зображення розміром 28 × 28 пікселів трансформується в одномірний вектор довжиною 784 елементи. Функцію цього шару можна описати як послідовне «розгортання» рядків пікселів у суцільний масив. Важливо зазначити, що даний шар не має параметрів для навчання і слугує лише засобом зміни структури вхідних даних. Після етапу розгортання модель включає два повнозв'язні шари tf.keras.layers.Dense. Перший шар складається зі 128 нейронів і формує приховане подання даних. Завершальний шар генерує вихідний вектор імовірнісних значень, сума яких дорівнює одиниці. Кожен елемент цього вектора відображає ймовірність того, що досліджуване зображення належить до певного класу рукописних цифр від нуля до дев'яти.

Перед початком навчання моделі необхідно визначити ще кілька налаштувань. Вони задаються під час етапу компіляції моделі.

Функція втрат (Loss function). Вона визначає, наскільки точно модель працює під час навчання. У процесі тренування ми прагнемо мінімізувати цю функцію, що дає змогу «спрямовувати» модель у правильному напрямку.

Оптимізатор (Optimizer). Він задає спосіб оновлення параметрів моделі на основі даних, які вона опрацьовує, та значення функції втрат.

Метрики (Metrics). Це показники, які використовуються для моніторингу як етапів навчання, так і тестування. У даному прикладі використовується метрика *accuracy*, яка відображає частку правильно класифікованих зображень.

На початковому етапі ми застосуємо оптимізатор стохастичного градієнтного спуску (SGD), ініціалізований із швидкістю навчання 0.1. Оскільки розв'язується задача багатокласової класифікації, доцільно використовувати функцію втрат типу *cross entropy*.

У подальшому бажано експериментувати з вибором оптимізатора та швидкістю навчання, щоб оцінити, як ці параметри впливають на точність побудованої моделі.

```
# Компіляція моделі
model.compile(
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
# Стохастичний градієнтний спуск
    loss="sparse_categorical_crossentropy",
# Функція втрат для багатокласової класифікації
    metrics=["accuracy"]
# Метрика точності
)
```

На цьому етапі можна переходити безпосередньо до навчання моделі. Суть процесу полягає у подачі навчальних прикладів (train_images та train_labels) до моделі, яка в результаті поступово формує відповідність між зображеннями та їхніми класами. Під час організації навчання важливо заздалегідь визначити такі параметри, як розмір пакета (batch size) та кількість епох (epochs), що відповідають числу повних проходів по набору даних MNIST.

Реалізувати оптимізацію функції втрат та навчання моделі на основі стохастичного градієнтного спуску можна за допомогою механізму GradientTape. Проте після виконання кроку компіляції існує простіший спосіб – використання методу fit, що надається класом Model. Цей метод дає змогу організувати процес тренування автоматично.

```
# Визначаємо розмір пакета та кількість епох для навчання
BATCH_SIZE = 64       # кількість прикладів у пакеті
EPOCHS = 5            # кількість повних проходів по датасету
```

```
# Запуск навчання моделі
model.fit(
    train_images,
    train_labels,
    batch_size=BATCH_SIZE,
    epochs=EPOCHS,
    validation_split=0.1
)

# Завершення сесії експерименту Comet (якщо використовується Comet ML)
comet_model_1.end()
```

У процесі навчання моделі виводяться значення функції втрат та метрики точності. За п’ять епох та швидкості навчання 0.01 повнозв’язна модель очікувано досягає точності близько 0.97 (97%) на навчальних даних.

Після завершення етапу навчання модель можна перевірити на нових даних, які не використовувалися під час тренування. У даному випадку тестовий набір зображень збережений у масиві test_images, а відповідні правильні мітки – у test_labels. Щоб оцінити якість роботи моделі, необхідно порівняти її передбачення з правильними класами. Для цього у TensorFlow передбачений метод evaluate, який запускає процес тестування та повертає значення функції втрат і вибраних метрик для тестових даних.

```
# Використовуємо метод evaluate для перевірки моделі на тестових даних
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)

# Виводимо результат точності на тестовій вибірці
print("Точність на тестових даних:", test_acc)
```

Зазвичай під час перевірки можна спостерігати, що точність на тестових даних виявляється нижчою, ніж на навчальних. Така різниця свідчить про явище **перенавчання**, коли модель машинного навчання демонструє хороші результати на даних, використаних для тренування, але працює гірше з новими прикладами.

Виникає закономірне питання: **якого рівня точності реально досягти, використовуючи цю базову повнозв’язну архітектуру?** Оскільки завдання класифікації рукописних цифр є відносно нескладним, цілком природно замислитися над можливостями подальшого удосконалення моделі для підвищення її ефективності.

Згорткова нейронна мережа (CNN) для розпізнавання рукописних цифр

Повнозв’язна модель дає змогу ознайомитися з основними етапами роботи з Keras, однак після перетворення зображення шаром Flatten його просторові зв’язки фактично не використовуються. Для задач класифікації зображень доцільніше застосувати згорткову нейронну мережу, яка виділяє локальні ознаки за допомогою згорткових фільтрів.

Згорткові нейронні мережі довели свою ефективність у розв’язанні завдань комп’ютерного зору, зокрема вони демонструють майже стовідсоткову точність на наборі даних MNIST. На цьому етапі побудуємо модель CNN, до складу якої входитимуть два згорткові шари з подальшими шарами підвибірки (pooling), а також два повнозв’язні шари. Завершальний вихідний рівень формуватиме ймовірнісний розподіл для десяти класів цифр (від 0 до 9). Архітектуру цієї мережі наведено нижче.

Архітектура згорткової нейронної мережі для MNIST

№	Шар	Параметри	Вихідні дані
1	Вхід	28 × 28 × 1 (зображення)	28 × 28 × 1
2	Conv2D	24 фільтри, ядро 3×3, ReLU	26 × 26 × 24
3	MaxPooling2D	вікно 2×2	13 × 13 × 24
4	Conv2D	36 фільтрів, ядро 3×3, ReLU	11 × 11 × 36
5	MaxPooling2D	вікно 2×2	5 × 5 × 36
6	Flatten	перетворення у вектор	900
7	Dense	128 нейронів, ReLU	128
8	Dense (вихідний)	10 нейронів, Softmax	10 класів (0–9)

Побудова згорткової нейронної мережі здійснюватиметься на тих самих навчальних і тестових вибірках, що й у попередньому прикладі з повнозв'язною мережею. Процедура організації навчання також залишиться подібною, однак цього разу будуть застосовані два нові типи шарів: keras.layers.Conv2D, який використовується для формування згорткових шарів, та keras.layers.MaxPool2D, що реалізує операцію підвибірки (*pooling*). Спираючись на параметри, наведені в описі архітектури, необхідно задати ці шари та сформувати модель CNN.

```
def build_cnn_model():
        cnn_model = tf.keras.Sequential([

            # Перший згортковий шар: 24 фільтри, ядро 3×3, активація ReLU
            tf.keras.layers.Conv2D(filters=24,         kernel_size=(3,3),
activation=tf.nn.relu),

            # Перший шар підвибірки (max pooling) з вікном 2×2
            tf.keras.layers.MaxPool2D(pool_size=(2,2)),

            # Другий згортковий шар: 36 фільтрів, ядро 3×3, активація ReLU
            tf.keras.layers.Conv2D(filters=36,         kernel_size=(3,3),
activation=tf.nn.relu),

            # Другий шар підвибірки (max pooling) з вікном 2×2
            tf.keras.layers.MaxPool2D(pool_size=(2,2)),

            # Перетворення у вектор
            tf.keras.layers.Flatten(),

            # Прихований повнозв'язний шар із 128 нейронами та активацією
            tf.keras.layers.Dense(128, activation=tf.nn.relu),

            # Вихідний шар: 10 нейронів для класифікації класів (Softmax)
            tf.keras.layers.Dense(10, activation=tf.nn.softmax)
        ])

        return cnn_model

    # Створення моделі
    cnn_model = build_cnn_model()

    # Ініціалізація моделі за допомогою пропускання одного прикладу
    cnn_model.predict(train_images[[0]])

    # Виведення структури моделі з описом усіх шарів
    print(cnn_model.summary())
```

Як і у попередньому випадку з повнозв'язною моделлю, для згорткової мережі

необхідно визначити функцію втрат, оптимізатор і метрики. Це виконується за допомогою методу compile, де можна обрати оптимізатор та встановити швидкість навчання.

```
# Ініціалізація нового експерименту в Comet ML
      comet_ml.init(project_name="lab1_part1_CNN")
      comet_model_2 = comet_ml.Experiment()

      # Компіляція моделі CNN:
      # - оптимізатор Adam зі швидкістю навчання 0.001
      # - функція втрат sparse_categorical_crossentropy для багатокласової
класифікації
      # - метрика оцінювання accuracy
      cnn_model.compile(
          optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
          loss="sparse_categorical_crossentropy",
          metrics=["accuracy"]
      )
```

Далі навчання моделі здійснюється методом fit із використанням API Keras.

```
# Навчання згорткової нейронної мережі (CNN)
      # Використовуються ті самі параметри batch_size та epochs, що й у
попередньому прикладі
      cnn_model.fit(
          train_images,
          train_labels,
          batch_size=BATCH_SIZE,
          epochs=EPOCHS,
          validation_split=0.1
      )

      # Завершення експерименту в Comet ML (опційно)
      # comet_model_2.end()
```

Після завершення тренування слід оцінити якість роботи моделі на тестових даних. Для цього застосовується метод evaluate, який повертає значення функції втрат і метрик на незалежній вибірці.

```
      # Оцінювання якості роботи моделі на тестових даних
      # Метод evaluate повертає значення функції втрат та точність
      класифікації
      test_loss, test_acc = cnn_model.evaluate(test_images, test_labels,
      verbose=2)

      # Виведення результату точності на тестовій вибірці
      print("Точність на тестових даних:", test_acc)
```

Після навчання можна отримати імовірнісні передбачення для нових зображень за допомогою методу predict:

```
      # Передбачення зображень тестового набору
      predictions = cnn_model.predict(test_images)
      # Результат прогнозу для першого зображення
      predictions[0]
```

Як видно, результат прогнозу являє собою масив із 10 чисел. Нагадаємо, що вихід моделі – це розподіл імовірностей по десяти класах цифр. Відповідно, кожне число описує рівень «впевненості» моделі в тому, що зображення належить до конкретної цифри.

Далі розглянемо той клас, для якого модель продемонструвала найбільшу впевненість при класифікації першого зображення з тестового набору.

```
# Визначення класу з максимальною ймовірністю для першого зображення
# тестового набору
prediction = np.argmax(predictions[0])

# Виведення передбаченого класу
print("Передбачена цифра для першого зображення:", prediction)
```

Отже, модель із найбільшою впевненістю класифікує це зображення як певну цифру. Щоб перевірити правильність передбачення, можна звернутися до тестової мітки, яка є справжнім значенням для цього зображення, та порівняти її з прогнозом моделі.

```
# Виведення справжньої мітки для першого зображення з тестового набору
print("Справжня мітка для цього зображення:", test_labels[0])

# Відображення самого зображення
plt.imshow(test_images[0, :, :, 0], cmap=plt.cm.binary)

# Логування зображення у Comet ML (опційно)
comet_model_2.log_figure(figure=plt)
```

Передбачення виявилося правильним. Тепер перейдемо до візуалізації результатів класифікації на наборі даних MNIST. Для цього побудуємо зображення з тестової вибірки разом із передбаченими для них класами, а також гістограму, що відображає розподіл імовірностей прогнозу для кожної з цифр.

```
# Вибір індексу зображення для візуалізації
image_index = 118  # приклад: 118-те зображення з тестового набору

# ---------------- Функція для візуалізації зображення --------------
def plot_image_prediction(i, predictions, true_labels, images, class_names=None):
    pred_array, true_label, img = predictions[i], true_labels[i], images[i]

    plt.grid(False)
    plt.xticks([])
    plt.yticks([])

    # Підтримка формату зображень (28x28 або 28x28x1)
    if img.ndim == 2:
        plt.imshow(img, cmap=plt.cm.binary)
    elif img.ndim == 3 and img.shape[-1] == 1:
        plt.imshow(img[:, :, 0], cmap=plt.cm.binary)
    else:
        plt.imshow(img)

    predicted_label = np.argmax(pred_array)
    confidence = 100 * np.max(pred_array)

    color = "blue" if predicted_label == true_label else "red"
    label_text = f"pred: {predicted_label} ({confidence:.1f}%)\ntrue: {true_label}"
    plt.xlabel(label_text, color=color, fontsize=10)

# ---------------- Функція для гістограми ----------------
def plot_value_prediction(i, predictions, true_labels):
    pred_array, true_label = predictions[i], int(true_labels[i])

    plt.grid(False)
    plt.xticks(range(10))
    plt.yticks([])
```

```
        bars = plt.bar(range(10), pred_array, color="gray")

        predicted_label = np.argmax(pred_array)
        bars[predicted_label].set_color("red")
        bars[true_label].set_color("blue")
        plt.ylim([0, 1.0])
        plt.xlabel("Ймовірності класів (0-9)")

    # ---------------- Використання ----------------
    plt.figure(figsize=(8, 4))

    # Ліва частина: саме зображення
    plt.subplot(1, 2, 1)
    plot_image_prediction(image_index,     predictions,     test_labels,
test_images)

    # Права частина: гістограма
    plt.subplot(1, 2, 2)
    plot_value_prediction(image_index, predictions, test_labels)

    plt.tight_layout()
    plt.show()

    # Логування у Comet ML (опційно)
    # comet_model_2.log_figure(figure=plt)
```

Можна також візуалізувати одразу кілька зображень разом із передбаченнями моделі. При цьому правильні мітки прогнозів відображаються синім кольором, а неправильні червоним. Поруч вказується рівень впевненості моделі у відсотках (від 0 до 100) для передбаченого класу. Варто звернути увагу, що іноді модель може демонструвати дуже високий рівень упевненості, навіть якщо прогноз є помилковим.

```
    # Виведення перших X зображень тестового набору разом із прогнозами
    моделі
    # Для правильних передбачень підпис відображається синім кольором, #
    для неправильних – червоним
    num_rows = 5     # кількість рядків у візуалізації
    num_cols = 4     # кількість стовпців
    num_images = num_rows * num_cols

    plt.figure(figsize=(2 * 2 * num_cols, 2 * num_rows))

    for i in range(num_images):
        # Ліва частина: зображення + передбачення
        plt.subplot(num_rows, 2 * num_cols, 2 * i + 1)
        plot_image_prediction(i, predictions, test_labels, test_images)

        # Права частина: гістограма ймовірностей
        plt.subplot(num_rows, 2 * num_cols, 2 * i + 2)
        plot_value_prediction(i, predictions, test_labels)

    plt.tight_layout()
    plt.show()

    # Логування результатів у Comet ML
    comet_model_2.log_figure(figure=plt)

    # Завершення експерименту
    comet_model_2.end()
```

Індивідуальні завдання

Для виконання лабораторної роботи використайте наведений вище приклад побудови згорткової нейронної мережі для класифікації рукописних цифр набору MNIST.

Відповідно до номера варіанта встановіть задані параметри навчання моделі: розмір пакета batch size, кількість епох та оптимізатор. Додатково виконайте індивідуальне завдання, пов'язане з дослідженням архітектури, параметрів навчання або результатів роботи згорткової нейронної мережі.

Базову архітектуру CNN, наведену в лабораторній роботі, можна використовувати як початкову. Залежно від індивідуального завдання дозволяється змінювати кількість згорткових шарів, кількість фільтрів, розміри ядер згортки, функції активації, кількість нейронів у повнозв'язному шарі, додавати шари Dropout, а також змінювати інші параметри моделі.

Під час проведення порівняльних експериментів необхідно змінювати лише досліджуваний параметр, залишаючи решту параметрів моделі та процесу навчання незмінними. Це дає змогу коректніше оцінити вплив досліджуваного компонента на результати роботи CNN.

Загальна вимога для всіх варіантів. Під час навчання згорткової нейронної мережі необхідно використати засіб для моніторингу та візуалізації процесу навчання відповідно до номера варіанта:

- **непарні варіанти** — **Comet ML**;
- **парні варіанти** — **TensorBoard**.

У вибраному середовищі необхідно відобразити та зберегти основні результати експерименту:

- значення функції втрат `loss`;
- значення метрики `accuracy`;
- графіки їх зміни в процесі навчання;
- основні параметри експерименту: `batch size`, кількість епох, оптимізатор, `learning rate`;
- за можливості — додаткові графіки або візуалізації, отримані під час виконання індивідуального завдання.

Для завдань, що передбачають порівняння кількох конфігурацій CNN, кожну конфігурацію слід запускати як окремий експеримент, щоб забезпечити можливість зіставлення отриманих результатів.

№ варіанта	Batch size	Кількість епох	Оптимізатор	Додаткове завдання
1	32	5	Adam	Дослідити вплив функцій активації `ReLU`, `LeakyReLU`, `tanh` та `sigmoid` у згорткових шарах CNN на точність класифікації. Порівняти отримані результати.
2	64	10	SGD	Порівняти результати навчання CNN із використанням функцій втрат

				`sparse_categorical_crossentropy` та `categorical_crossentropy`.
3	128	15	RMSprop	Побудувати **confusion matrix** для результатів класифікації CNN. Визначити класи цифр, які модель найчастіше плутає між собою.
4	256	20	Adam	Дослідити вплив кількості фільтрів у згорткових шарах CNN. Порівняти щонайменше дві конфігурації, наприклад `16–32` та `32–64` фільтри.
5	40	5	SGD	Візуалізувати помилково класифіковані тестові зображення. Для кожного прикладу вказати правильний клас, прогноз CNN та рівень упевненості моделі.
6	54	10	Adam	Дослідити вплив кількості нейронів у повнозв'язному шарі CNN. Порівняти щонайменше три конфігурації, наприклад 64, 128 та 256 нейронів.
7	75	15	RMSprop	Вивести приклади правильно та неправильно класифікованих тестових зображень із зазначенням передбаченого класу та рівня впевненості CNN.
8	100	20	Adam	Побудувати **confusion matrix у вигляді heatmap**. Проаналізувати розподіл правильних і помилкових передбачень для різних класів.
9	32	10	RMSprop	Знайти та візуалізувати помилкові передбачення, для яких CNN продемонструвала найбільший рівень упевненості. Проаналізувати можливі причини таких помилок.
10	64	15	Adam	Дослідити вплив функцій активації у згорткових шарах CNN. Порівняти результати щонайменше для трьох функцій активації.
11	128	20	SGD	Дослідити вплив розміру ядра згортки на результати CNN. Порівняти архітектури з ядрами `3×3` і `5×5`.
12	256	5	Adam	Побудувати confusion matrix та визначити для кожного класу кількість правильних і помилкових передбачень. Визначити класи з найбільшою кількістю помилок.
13	40	15	RMSprop	Візуалізувати **карти активацій (feature maps)** одного зі згорткових шарів CNN для декількох тестових зображень. Проаналізувати, які ознаки зображення виділяють окремі фільтри.
14	54	20	SGD	Порівняти результати навчання CNN із двома функціями втрат. Побудувати та порівняти графіки зміни `loss` і `accuracy` у процесі навчання.
15	75	5	Adam	Вивести та проаналізувати приклади правильно і неправильно класифікованих зображень. Для кожного прикладу вказати прогноз CNN та рівень упевненості моделі.

16	100	10	RMSprop	Дослідити вплив шару `Dropout` на роботу CNN. Порівняти модель без `Dropout` і модель із `Dropout`, проаналізувавши точність на навчальних і тестових даних та можливі ознаки перенавчання.

Для завдань, у яких використовується функція втрат categorical_crossentropy, мітки класів необхідно попередньо перетворити у формат one-hot encoding.

У завданнях, що передбачають побудову карт активацій, необхідно використати вихідні значення одного або декількох згорткових шарів навченої CNN.

Після завершення роботи необхідно оцінити модель на тестовій вибірці та сформулювати короткі висновки щодо ефективності реалізованої CNN і впливу досліджуваного параметра.

Зміст звіту

Звіт з лабораторної роботи повинен містити:

1. Титульний аркуш із зазначенням назви лабораторної роботи, номера варіанта, прізвища студента та навчальної групи.
2. Індивідуальне завдання із заданими параметрами навчання: `batch size`, кількість епох, оптимізатор та додаткове завдання.
3. Програмний код реалізації CNN, що включає підготовку даних, побудову, навчання та тестування моделі, а також виконання індивідуального завдання.
4. Структура моделі CNN (`model.summary()`) та короткий опис її основних шарів.
5. Результати навчання і тестування, що містять:
   - значення `loss` та `accuracy`;
   - точність на тестовій вибірці;
   - приклади результатів класифікації.
6. Результати моніторингу експерименту в TensorBoard або Comet ML відповідно до номера варіанта:
   - графіки loss та accuracy;
   - основні параметри експерименту;
   - скріншот сторінки або панелі з результатами;
   - для порівняльних завдань –9 результати кількох запусків.
7. Результати виконання індивідуального завдання – confusion matrix, heatmap, карти активацій, приклади правильних/помилкових прогнозів або результати порівняння досліджуваних конфігурацій CNN.
8. Аналіз результатів та висновки щодо точності моделі, впливу досліджуваного параметра та ефективності реалізованої CNN.